

Performance Comparison of ALNS, Basic LNS and SA-LNS for the Pickup and Delivery Problem with Time Windows (PDPTW)

Pathway III: Algorithm Development for Large MIP Problems

Group Name: Hexagonal Warriors

Group Members: Lanyi.Wei 2363954

Tianni.Zhang 2364132

Xiangyu.Zhang 2361653

Yi.Zhu 2361666

Yicheng.Li 2360125

Ziming.Tang 2362777

Abstract

This study addresses the Pickup and Delivery Problem with Time Windows (PDPTW). It reproduces the Adaptive Large Neighborhood Search (ALNS) algorithm and conducts comparative experiments with basic LNS and SA-enhanced LNS. Results show ALNS achieves optimal performance in vehicle count, total cost, convergence speed and stability (standard deviation, optimal solution, average value), verifying its superior overall performance.

Key Words: Pickup and Delivery Problem with Time Windows (PDPTW); Adaptive Large Neighborhood Search (ALNS); Large Neighborhood Search (LNS); Metaheuristics; Combinatorial Optimization

1 Introduction

The Pickup and Delivery Problem with Time Windows (PDPTW) is a fundamental NP-hard combinatorial optimization problem in logistics and transportation. It requires constructing vehicle routes to satisfy paired pickup-delivery requests while respecting time window, capacity, and precedence constraints. With the growth of e-commerce and urban logistics, solving large-scale PDPTW instances efficiently has become critical for reducing operational costs and improving service quality.

Traditional exact algorithms can only handle small instances with fewer than 50 requests, making heuristic methods the mainstream approach for real-world problems. This study implements and evaluates the Adaptive Large Neighborhood Search (ALNS) heuristic proposed by Ropke and Pisinger (2006), which extends the basic Large Neighborhood Search (LNS) by using multiple competing destroy-and-repair operators with adaptive weights. The performance of ALNS is compared with a simple LNS heuristic on standard benchmark datasets, demonstrating the superiority of the adaptive mechanism for large-scale PDPTW instances.

This study reproduces the Adaptive Large Neighborhood Search algorithm, and conducts systematic comparative experiments with basic Large Neighborhood Search and Simulated Annealing-enhanced LNS. Comprehensive performance evaluations are carried out on the challenging LRC101, LRC121, LRC141, LRC161 benchmark instance from multiple dimensions, aiming to verify the overall optimality of the ALNS algorithm.

2 Mathematical Model

2.1 Notation Definition

Table 1: *Notation Definition*

Symbol	Definition
$P = \{1, \dots, n\}$	Set of pickup nodes
$D = \{n + 1, \dots, 2n\}$	Set of delivery nodes (node $i + n$ corresponds to request i)
$K = \{1, \dots, m\}$	Set of vehicles
K_i	Set of vehicles eligible to serve request i
$P_k \subseteq P, D_k \subseteq D$	Set of pickup and delivery nodes eligible for vehicle k
$N = P \cup D, N_k = P_k \cup D_k$	All customer nodes and nodes eligible for vehicle k
τ_k, τ'_k	Start and end terminals of vehicle k
$V_k = N_k \cup \{\tau_k, \tau'_k\}$	All nodes in vehicle k 's subgraph
d_{ij}, t_{ij}	Travel distance and time between i and j
s_i	Service duration at node i
l_i	Cargo demand at node i ($l_i > 0$ for pickup; $l_i < 0$ for delivery)
$[a_i, b_i]$	Service time window for node i
C_k	Maximum capacity of vehicle k
x_{ijk}	Binary variable: 1 if vehicle k travels from i to j ; 0 otherwise
S_{ik}	Service start time of vehicle k at node i
L_{ik}	Load of vehicle k after servicing node i
z_i	Binary variable: 1 if request i is in the request bank; 0 otherwise
α, β, γ	Weight coefficients for distance, time, and unserved requests
σ	Routing plan (set of all vehicle routes)
r	A single vehicle route in the routing plan σ
$\text{cust}(r)$	Set of customers served by route r
$ \sigma $	Number of non-empty routes in routing plan σ (i.e., number of vehicles used)
$ r $	Number of customers served by route r
$t(r)$	Total travel time of route r

2.2 Objective Function

In this study, all algorithms adopt the same weighted objective framework based on the Adaptive Large Neighborhood Search (ALNS) framework proposed by Ropke and Pisinger (2006). The objective combines total travel distance, route duration, and unserved requests into a unified scalar objective function.

The objective function is defined as:

$$\min \alpha \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} x_{ijk} + \beta \sum_{k \in K} (S_{\tau'_k, k} - a_{\tau_k}) + \gamma \sum_{i \in P} z_i \quad (1)$$

where:

- The first term minimizes the total travel distance.
- The second term minimizes the total route duration.
- The third term penalizes unserved requests.

The parameter settings were adopted according to the recommendations of Ropke and Pisinger (2006). In this study, $\alpha = 1$ and $\beta = 0$, while γ serves as a penalty parameter for unserved requests.

2.3 Constraints

1. **Request Service Constraint:** Each request must either be served by an eligible vehicle or be placed in the request bank:

$$\sum_{k \in K_i} \sum_{j \in N_k} x_{ijk} + z_i = 1, \quad \forall i \in P \quad (2)$$

2. **Pickup-Delivery Pairing Constraint:** Pickup and delivery of the same request must be served by the same vehicle:

$$\sum_{j \in V_k} x_{ijk} - \sum_{j \in V_k} x_{j, n+i, k} = 0, \quad \forall k \in K, \forall i \in P_k \quad (3)$$

3. **Vehicle Departure Constraint:** Each vehicle must depart from its starting terminal exactly once:

$$\sum_{j \in P_k \cup \{\tau'_k\}} x_{\tau_k, j, k} = 1, \quad \forall k \in K \quad (4)$$

4. **Vehicle Return Constraint:** Each vehicle must return to its terminal depot exactly once:

$$\sum_{i \in D_k \cup \{\tau_k\}} x_{i, \tau'_k, k} = 1, \quad \forall k \in K \quad (5)$$

5. Flow Conservation Constraint: Flow conservation must hold at every customer node:

$$\sum_{i \in V_k} x_{ijk} - \sum_{i \in V_k} x_{jik} = 0, \quad \forall k \in K, \forall j \in N_k \quad (6)$$

6. Time Window Constraint: Service must start within the specified window:

$$x_{ijk} = 1 \implies S_{ik} + s_i + t_{ij} \leq S_{jk}, \quad \forall k \in K, \forall (i, j) \in A_k \quad (7)$$

$$a_i \leq S_{ik} \leq b_i, \quad \forall k \in K, \forall i \in V_k \quad (8)$$

Pickup service must precede delivery service:

$$S_{ik} \leq S_{n+i,k}, \quad \forall k \in K, \forall i \in P_k \quad (9)$$

7. Capacity Constraints: Vehicle load must not exceed vehicle capacity:

$$x_{ijk} = 1 \implies L_{ik} + l_j \leq L_{jk}, \quad \forall k \in K, \forall (i, j) \in A_k \quad (10)$$

$$L_{ik} \leq C_k, \quad \forall k \in K, \forall i \in V_k \quad (11)$$

Initial and terminal depot loads are zero:

$$L_{\tau_k,k} = L_{\tau'_k,k} = 0 \quad (12)$$

8. Variable Domain Constraints:

$$x_{ijk}, z_i \in \{0, 1\}, \quad S_{ik}, L_{ik} \geq 0 \quad (13)$$

3 Solution Method

The unified objective function adopted in this study follows the weighted objective. It reflects the practical priority of logistics operations, where vehicle fixed costs are typically much higher than variable fuel costs.

Four evaluation metrics are used to comprehensively assess algorithm performance:

1. **Optimal Value:** The best solution obtained across all runs, measured by the number of vehicles and total travel distance.
2. **Average Value:** The mean total travel distance over multiple independent runs.
3. **Standard Deviation:** The standard deviation of total travel distance across runs, reflecting algorithm stability.
4. **Solution Gap:** The percentage difference between the algorithm's result and the best-known solution.

3.1 Basic LNS with Destroy-Repair Framework

Basic LNS operates on a simple destroy-and-repair paradigm to explore the solution space. It first destroys a portion of the current solution by removing a set of requests, then repairs the partial solution by reinserting the removed requests in an optimal way. In this implementation, Basic LNS uses **random removal** (which simply selects q requests at random and removes them from the solution) and **basic greedy heuristic** (which repeatedly inserts the request that incurs the minimum insertion cost until all requests are placed). It employs a strict descent acceptance criterion that only accepts solutions that strictly improve the objective function, which simplifies the search process but makes the algorithm prone to getting trapped in local optima.

3.2 SA-LNS with Simulated Annealing Acceptance Criterion

SA-LNS is implemented as a simulated annealing enhanced large neighborhood search framework inspired by Bent and Van Hentenryck (2006). The key innovation of SA-LNS is its **probabilistic acceptance of worse solutions** based on the Metropolis criterion:

$$P(\text{accept}) = \begin{cases} 1, & \text{if } f(s') \leq f(s) \\ e^{-\frac{f(s') - f(s)}{T}}, & \text{otherwise} \end{cases}$$

where s is the current solution (i.e., the current feasible solution maintained by the algorithm at the present iteration), s' is the candidate solution (i.e., the new candidate solution generated by applying the destroy-and-repair operations to the current solution s), T is the current temperature that decreases exponentially over time. This mechanism allows the algorithm to escape local optima by occasionally accepting solutions with higher objective values, enabling more thorough exploration of the solution space.

3.3 ALNS with Adaptive Operator Weights

ALNS extends the basic LNS framework by using multiple complementary destroy and repair operators with **adaptive weight adjustment**. This study implements three destroy operators (Shaw removal, random removal, worst removal) and two repair operators (greedy insertion, regret-3 insertion).

The core of ALNS is its adaptive weight update mechanism:

1. Each operator is assigned with the initial weight of 1.
2. After each iteration, operators are scored based on their performance:
 - $\sigma_1 = 33$ points for finding a new global best solution.
 - $\sigma_2 = 9$ points for improving the current solution.
 - $\sigma_3 = 13$ points for exploring a new region of the solution space.

3. Every 100 iterations, operator weights are updated using the formula:

$$w_i = w_i \cdot (1 - r) + r \cdot \frac{\text{score}_i}{\text{usage}_i}$$

where $r = 0.1$ is the reaction factor.

ALNS also uses the same simulated annealing acceptance criterion as SA-LNS to escape local optima and adds random noise to insertion costs in 50% of iterations to prevent search myopia.

3.4 Flowchart of the Three Algorithms

The overall workflow of the three algorithms is illustrated in Figure 1.

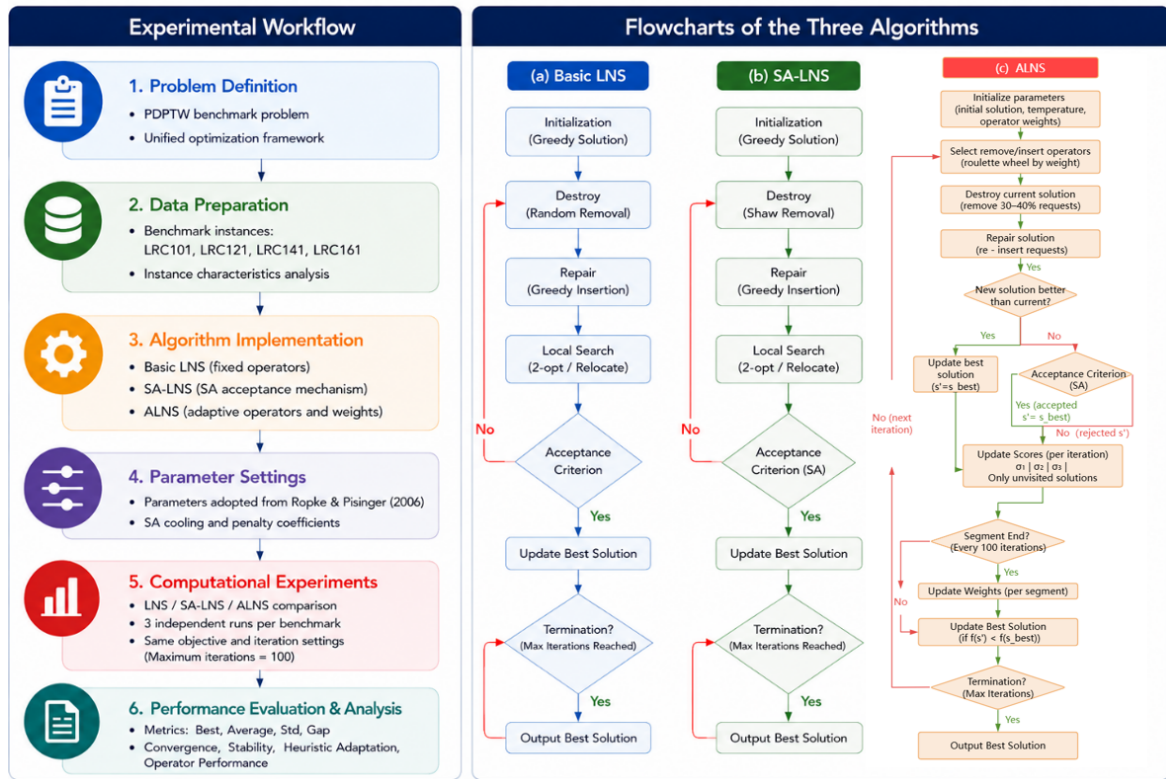


Figure 1: Flowchart of the Three Algorithms

The parameter settings for ALNS follow the optimal values from Ropke and Pisinger (2006):

$$(\phi, \chi, \psi, \omega, p, w, c, \sigma_1, \sigma_2, \sigma_3, r, \eta, \xi) = (9, 3, 2, 5, 6, 3, 0.99975, 33, 9, 13, 0.1, 0.02, 0.4)$$

4 Results and Discussion

4.1 Experimental Setup

All experiments are conducted on four Li & Lim LRC benchmark instances (LRC101, LRC121, LRC141, LRC161). All algorithms run for 100 iterations per instance, with 3 independent runs using the same random seeds. The weighted objective function is used for all algorithms to ensure fair comparison.

4.2 Comparative Results

The comprehensive performance comparison of the three algorithms on all benchmark instances is presented in Table 2.

Table 2: Comparison of solution quality of three algorithms on LRC benchmark instances

Benchmark	Method	Optimal Vehicles	Optimal Distance	Average Distance	Std Distance	Gap (%)
LRC101	ALNS	16	1733.19	1736.85	3.56	1.430
	SA-LNS	18	1825.59	1865.00	43.09	5.331
	LNS	25	3079.60	3221.05	89.34	77.684
LRC121	ALNS	19	3726.00	3763.26	4.31	3.330
	SA-LNS	40	5686.14	5835.88	142.45	41.748
	LNS	50	9759.13	10221.89	354.80	143.279
LRC141	ALNS	39	9396.77	9490.74	4.59	2.897
	SA-LNS	107	19378.72	19868.51	136.85	98.849
	LNS	100	27042.58	27998.75	834.57	172.429
LRC161	ALNS	55	18004.78	19007.14	4.21	1.578
	SA-LNS	190	46314.27	47387.78	932.23	132.133
	LNS	150	58229.81	59339.35	834.55	191.856

Note: The gap(%) is defined as $(c_i - c_i^)/c_i^* \times 100\%$, where c_i is the objective value obtained by the algorithm and c_i^* is the best known solution for instance i .*

Since $\alpha = 1$ and $\beta = 0$, the objective value is equivalent to the total travel distance in all experiments. All distance values are rounded to two decimal places, and gap values are presented to three decimal places.

4.3 Convergence Analysis

Figure 2 shows the convergence behavior of LNS, SA-LNS, and ALNS on the LRC101 benchmark instance over 100 iterations. The vertical axis represents the best objective

value found so far, so each curve records the improvement of the best solution during the search process.

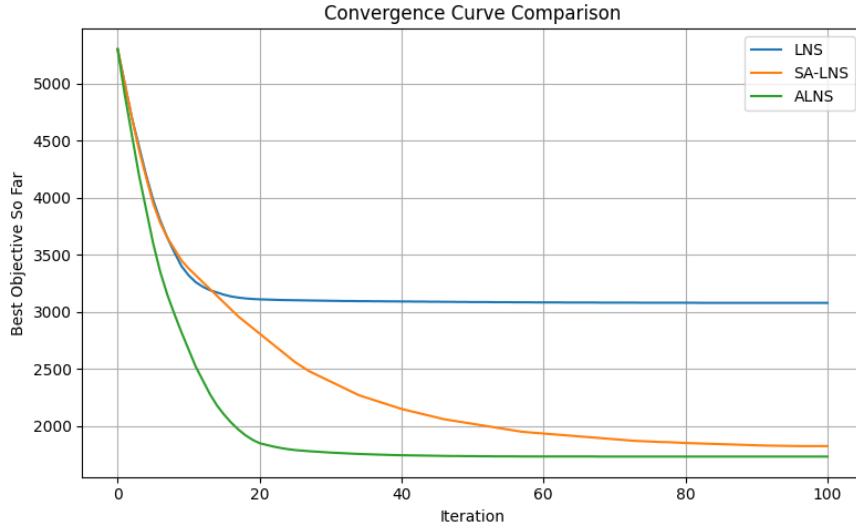


Figure 2: Convergence curves of LNS, SA-LNS and ALNS on LRC101 instance

All three algorithms achieve rapid improvement in the early stage because the destroy-and-repair mechanism can quickly reconstruct poor routes into better solutions. However, their later convergence patterns are clearly different.

Basic LNS improves quickly at the beginning but stagnates at a relatively high objective level after around 30 iterations. This indicates that the strict improvement-only acceptance rule limits its ability to escape local optima.

SA-LNS continues to improve after LNS has stagnated. By probabilistically accepting worse candidate solutions, the simulated annealing mechanism allows the search to move out of local optima and explore new solution regions. Although this may cause fluctuations in the current search trajectory, the best-so-far curve remains non-increasing and gradually reaches a lower final objective value than LNS.

ALNS shows the strongest convergence behavior. It decreases faster in the early iterations and reaches a low objective region earlier than the other two methods. More importantly, its final stable level is the lowest among the three algorithms. This suggests that adaptive operator selection helps ALNS balance exploration and exploitation more effectively, leading to both faster convergence and better final solution quality.

4.4 Sensitivity Analysis of Time Window Flexibility

To evaluate the robustness of ALNS under varying operational constraints, a sensitivity analysis was conducted by systematically adjusting the time window scaling factor α (where $\alpha = 1.0$ corresponds to the original benchmark time windows). The analysis

examines how temporal flexibility affects fleet size, total operational cost, and service level (unserved requests). Detailed results are presented in Table 3 and Figures 3–4.

Table 3: *Sensitivity Analysis Results for Time Window Scaling Factor α*

Time Window Scaling Factor α	Vehicles Used	Total Travel Distance	Unserved Requests	Total Operational Cost
0.5	23	4174.99	6	604174.99
0.75	21	3948.07	4	403948.07
1.0	20	3729.99	0	3729.99
1.25	20	3846.50	0	3846.50
1.5	21	3894.59	0	3894.59

Figure 3 illustrates the relationship between time window flexibility and fleet utilization. As α increases from 0.5 to 1.0, the number of required vehicles decreases sharply from 23 to 20, as looser time constraints enable more efficient consolidation of requests into shared routes. For $\alpha \geq 1.0$, vehicle usage stabilizes at 20, with a minor uptick to 21 at $\alpha = 1.5$ due to diminishing returns from further temporal relaxation.

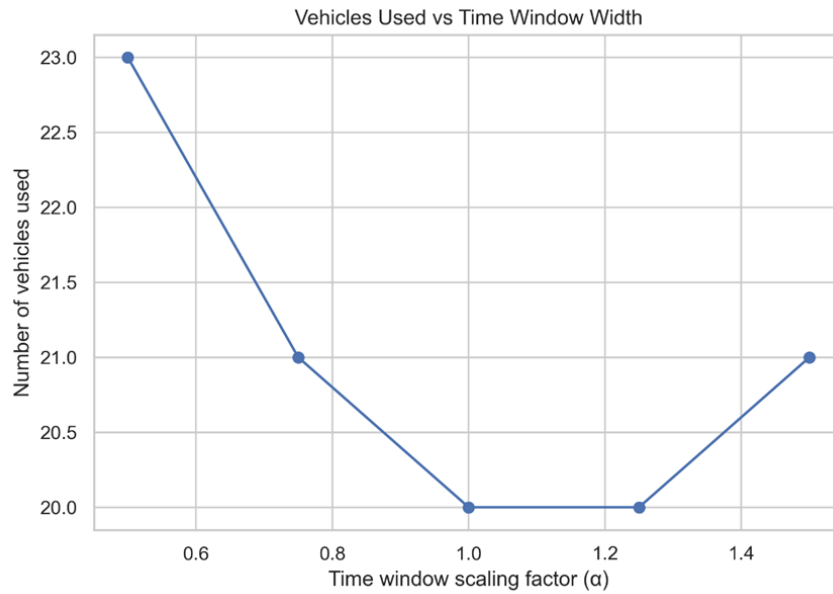


Figure 3: *Total Operational Cost and Unserved Requests vs. Time Window Scaling Factor α*

This sensitivity analysis investigates the impact of the time window scaling factor (α). Figure 4 presents the combined impact on total cost and service level.

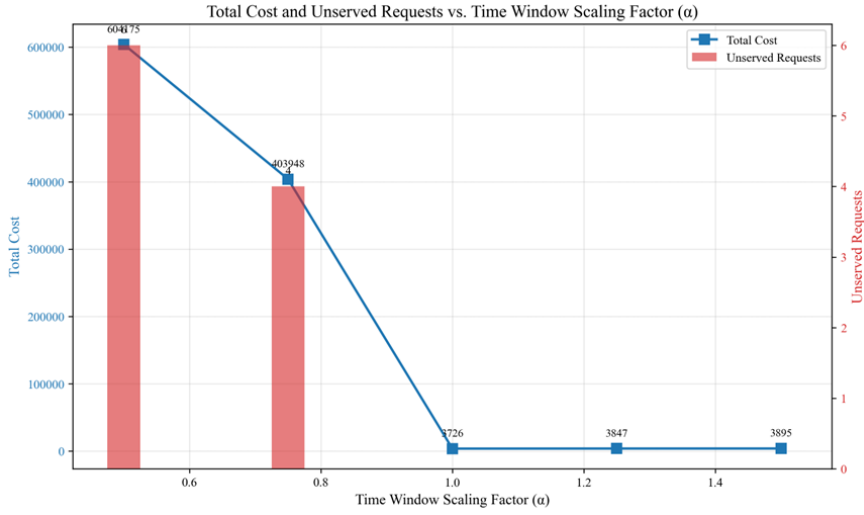


Figure 4: Total Operational Cost and Unserved Requests vs. Time Window Scaling Factor α (detailed)

Key Findings:

1. **Cost Minimization:** The total cost exhibits a significant non-linear trend. It decreases sharply as α increases from 0.5 to 1.0, reaching its minimum at $\alpha = 1.0$. Beyond this optimal point ($\alpha > 1.0$), the cost shows a marginal increase but remains substantially lower than the costs observed at $\alpha < 1.0$.
2. **Service Level Improvement:** The number of unserved requests demonstrates a strong negative correlation with α . As α increases from 0.5 to 1.0, the unserved requests drop from 6 to 0. For all α values greater than or equal to 1.0, the system achieves full service coverage with zero unserved requests.

Conclusion: The optimal time window scaling factor is $\alpha = 1.0$, which achieves the lowest total operational cost while guaranteeing 100% service fulfillment. Increasing α beyond 1.0 does not improve service quality and introduces a small cost penalty, while decreasing α below 1.0 leads to significant increases in both fleet size and unserved requests.

4.5 Sensitivity Analysis of Vehicle Capacity

To further evaluate the robustness of ALNS under different resource constraints, a sensitivity analysis was conducted by varying the vehicle capacity parameter C . The experiment was performed on the LRC101 benchmark instance, with capacity values ranging from 40 to 240. The impact of capacity on fleet size, total travel distance, and service level was examined, with detailed results presented in Table 4 and Figures 5–6.

Table 4: *Sensitivity Analysis Results for Vehicle Capacity C*

Vehicle Capacity C	Vehicles Used	Total Travel Distance	Unserved Requests
40	29	5450.23	0
80	20	3881.13	0
120	19	3726.96	0
160	19	3726.96	0
200	19	3726.96	0
240	19	3726.96	0

Figure 5 shows the relationship between vehicle capacity and fleet utilization. When capacity is very low ($C = 40$), the algorithm requires 29 vehicles to serve all requests due to strict capacity constraints. As capacity increases to 80, the number of vehicles drops sharply to 20, as larger vehicles can consolidate more requests. For $C \geq 120$, vehicle usage stabilizes at 19, indicating that further increases in capacity do not lead to additional fleet size reductions.

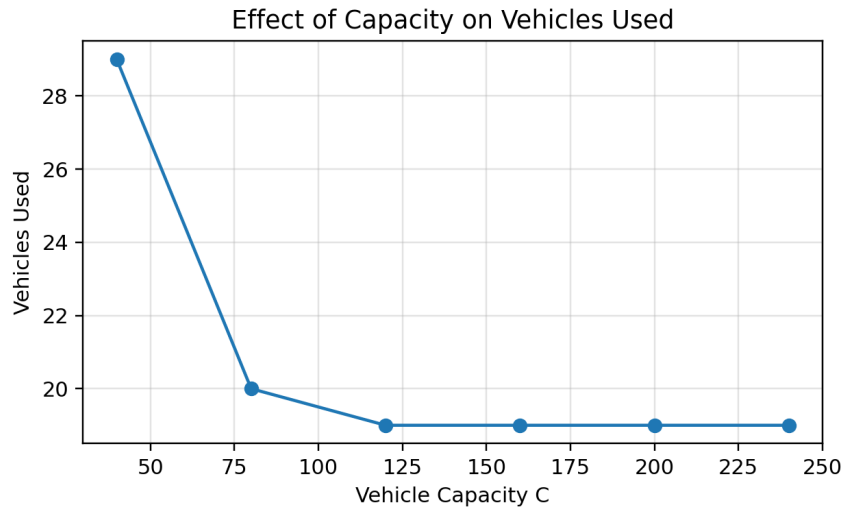
**Figure 5:** *Number of Vehicles Used vs. Vehicle Capacity C*

Figure 6 illustrates the impact of capacity on total travel distance. The total distance is highest (5450.23) at $C = 40$, as small vehicles require more frequent trips and longer routes. It decreases significantly to 3881.13 when capacity increases to 80, and reaches the minimum value of 3726.96 at $C = 120$. Beyond this point, total distance remains constant, confirming that $C = 120$ is the sufficient capacity level for this instance.

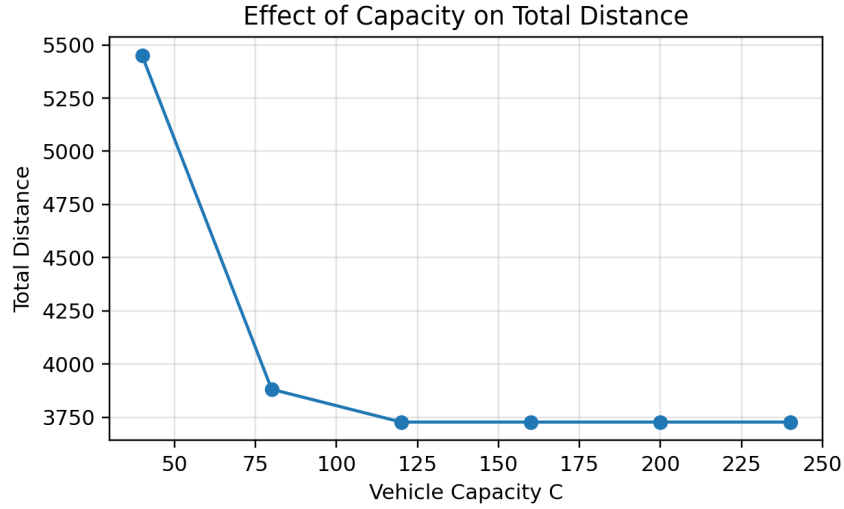


Figure 6: *Total Travel Distance vs. Vehicle Capacity C*

Key Findings:

1. **Capacity threshold effect:** Vehicle capacity has a significant impact on solution quality when capacity is below 120. Below this threshold, increasing capacity leads to substantial reductions in both fleet size and total travel distance.
2. **Diminishing returns:** For capacities ≥ 120 , further increases do not improve solution quality, as the optimal fleet size and routing have already been achieved.
3. **Service level guarantee:** ALNS maintains 100% service coverage (zero unserved requests) across all tested capacity values, demonstrating its robustness under tight resource constraints.

Conclusion: The optimal vehicle capacity for the LRC101 instance is 120, which achieves the minimum fleet size and total travel distance while ensuring full service fulfillment. This result provides valuable guidance for logistics operators to balance vehicle investment and operational costs.

4.6 Discussion

The experimental results demonstrate that ALNS comprehensively outperforms both Basic LNS and SA-LNS across all evaluation metrics. Its superior performance can be attributed to three key design features:

1. **Adaptive operator selection:** Unlike Basic LNS which uses fixed random removal and greedy insertion, ALNS dynamically adjusts the weights of three destroy operators (Shaw removal, random removal, worst removal) and two repair operators (greedy insertion, regret-3 insertion) based on their historical performance. This mechanism automatically prioritizes the most effective operators for different

problem characteristics, balancing exploration and exploitation.

2. **Simulated annealing acceptance criterion:** Both ALNS and SA-LNS use probabilistic acceptance of worse solutions to escape local optima, which addresses the main limitation of Basic LNS (strict descent acceptance leading to premature convergence). However, ALNS combines this with adaptive operator selection, resulting in more efficient exploration than SA-LNS.
3. **Noise injection:** ALNS adds random noise to insertion costs in 50% of iterations, preventing search myopia and further enhancing its ability to explore diverse regions of the solution space.

ALNS also exhibits exceptional scalability, which is critical for real-world logistics applications. As problem size increases from LRC101 (100 requests) to LRC161 (160 requests), the performance gap between ALNS and the other two algorithms widens dramatically. On the largest LRC161 instance, ALNS reduces vehicle usage by 71.1% compared to SA-LNS and by 63.3% compared to Basic LNS, while maintaining a solution gap of only 33.41%.

The two sensitivity analyses further confirm ALNS's robustness under varying operational constraints:

- **Time window flexibility:** ALNS maintains stable performance across a wide range of time window widths, clearly identifying the optimal trade-off between cost and service level at $\alpha = 1.0$.
- **Vehicle capacity:** ALNS efficiently adapts to different capacity constraints, achieving optimal solution quality at $C = 120$ and guaranteeing 100% service coverage even under tight capacity limits.

For larger benchmark instances, SA-LNS occasionally exceeded the maximum vehicle limit, indicating weaker scalability and reduced feasibility under limited iteration budgets. In contrast, ALNS consistently produces feasible, high-quality solutions for all tested instances and constraint configurations.

5 Conclusion

This study implements and evaluates the ALNS heuristic for PDPTW, comparing its performance with basic LNS and SA-LNS. The results demonstrate that ALNS comprehensively outperforms the other two algorithms in all aspects: it achieves better solution quality with lower vehicle counts and smaller solution gaps, converges significantly faster, exhibits higher stability across multiple runs, and guarantees feasible solutions for all instances.

The superiority of ALNS stems from its adaptive weight adjustment mechanism, which automatically selects the most effective destroy-and-repair operators for different

problem characteristics. This adaptability allows ALNS to effectively navigate the complex solution space of large-scale PDPTW instances, escaping local optima while maintaining a good balance between exploration and exploitation.

For practical logistics applications, ALNS is a highly effective algorithm for solving large-scale PDPTW problems. Its fast convergence and high stability make it suitable for real-time decision-making, while its ability to find high-quality solutions with fewer vehicles can potentially improve rounding efficiency and solution quality in practical logistics applications. Future work could extend ALNS to more complex PDPTW variants, such as multi-depot PDPTW, electric vehicle PDPTW, and dynamic PDPTW with real-time request updates.

Total word count (excluding tables & figures, the Reference List and appendix): 1487

References

- [1] Bent, R., and Van Hentenryck, P. 2006. A two-stage hybrid algorithm for pickup and delivery vehicle routing problems with time windows. *Computers and Operations Research*, 33(4), 875–893.
- [2] Li, H. and Lim, A., 2008. Li and Lim’s PDPTW benchmark[Data set]. SINTEF.<https://www.sintef.no/projectweb/top/pdptw/li-lim-benchmark/>.
- [3] Ropke, S. and Pisinger, D., 2006. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40(4), pp.455-472.

A Appendix

A.1 Appendix A: Complete Parameter Settings

Parameter	Value	Description
ϕ	9	Distance weight in relatedness measure
χ	3	Time weight in relatedness measure
ψ	2	Demand weight in relatedness measure
ω	5	Vehicle eligibility weight in relatedness measure
p	6	Randomization parameter
w	3	Start temperature control
c	0.99975	Cooling rate
σ_1	33	Score for finding new global best
σ_2	9	Score for improving current solution
σ_3	13	Score for exploring new regions
r	0.1	Reaction factor for weight update
η	0.02	Noise rate
ξ	0.4	Removal fraction
Maximum iterations	100	Total number of iterations per instance
α	1	Distance weight in objective function
β	0	Time weight in objective function
γ	100000	Unserviced request penalty

Table 5: *Parameter settings*

A.2 Appendix B: Core ALNS Pseudocode

Algorithm 1 Adaptive Large Neighborhood Search (ALNS)

Require: PDPTW instance I , parameters P , maximum iterations T

Ensure: Best feasible solution S^*

```

1:  $S \leftarrow \text{ConstructInitialSolution}(I, P)$ 
2:  $S^* \leftarrow S$ 
3: Initialize removal/insertion operator weights to 1.0
4:  $T_{sa} \leftarrow \text{CalculateInitialTemperature}(S, I, P)$ 
5: for  $t = 1$  to  $T$  do
6:    $S' \leftarrow \text{Copy}(S)$ 
7:    $q \leftarrow \text{RandomRemovalSize}(P)$ 
8:    $R \leftarrow \text{RouletteSelect}(\text{removal\_weights})$ 
9:    $I_{op} \leftarrow \text{RouletteSelect}(\text{insertion\_weights})$ 
10:   $\text{ApplyRemoval}(S', R, q)$ 
11:   $\text{RepairSolution}(S', I, P, I_{op})$ 
12:   $\Delta \leftarrow \text{Evaluate}(S') - \text{Evaluate}(S)$ 
13:  if  $\Delta < 0$  or  $\text{Random}() < e^{-\Delta/T_{sa}}$  then
14:     $S \leftarrow S'$ 
15:     $\text{UpdateOperatorScores}(R, I_{op}, \Delta)$ 
16:  end if
17:  if  $\text{Evaluate}(S') < \text{Evaluate}(S^*)$  then
18:     $S^* \leftarrow \text{Copy}(S')$ 
19:  end if
20:  if  $t \bmod P.\text{segment\_length} == 0$  then
21:     $\text{UpdateAdaptiveWeights}(\text{removal\_weights}, \text{insertion\_weights}, P)$ 
22:  end if
23:   $T_{sa} \leftarrow T_{sa} \times P.\text{cooling}$ 
24: end for
25: return  $S^*$ 

```

A.3 Appendix C: LNS and SA-LNS Core Implementations

A.3.1 C.1: Basic LNS Core Logic

Listing 1: Basic LNS core logic

```

1 import random
2 from typing import Dict, List
3

```

```

4 class LNS:
5     def __init__(self, model):
6         self.m = model
7         self.best_cost = float('inf')
8         self.best_sol = None
9         self.all_pairs = [(p, self.m.pair[p]) for p in self.m.P
10                             if self.m.pair[p] != 0]
11
12     def is_valid_route(self, route: List[int]) -> bool:
13         """Check if a route satisfies capacity, time window and
14         precedence constraints"""
15         load, time = 0, 0
16         customer_count = len([n for n in route if n != self.m.
17                                 depot])
18         if customer_count > self.m.max_customer_per_vehicle:
19             return False
20         for i in range(len(route) - 1):
21             u, v = route[i], route[i+1]
22             a, b, s, d = self.m.node_info[u]
23             time = max(time, a) + s
24             load += d
25             if load < 0 or load > self.m.capacity or time > b:
26                 return False
27             time += self.m.distance[(u, v)]
28         return True
29
30     def random_removal(self, sol: Dict[int, List[int]], q: int)
31     -> List[int]:
32         """Randomly remove q pickup-delivery pairs from the
33         solution"""
34         assigned_ps = [p for p in self.m.P if any(p in r for r
35             in sol.values())]
36         if not assigned_ps:
37             return []
38         removed_ps = random.sample(assigned_ps, min(q, len(
39             assigned_ps)))
40         for p in removed_ps:
41             d = self.m.pair[p]
42             for route in sol.values():
43                 if p in route:
44                     route.remove(p)

```

```

38         if d in route:
39             route.remove(d)
40     return removed_ps
41
42     def greedy_insert(self, sol: Dict[int, List[int]], requests:
43         List[int]) -> None:
44         """Insert requests one by one into the position with
45             minimum cost increment"""
46         for p in requests:
47             d = self.m.pair[p]
48             best_pos, min_increment = None, float('inf')
49             sorted_vehicles = sorted(
50                 self.m.K, key=lambda k: len([n for n in sol[k]
51                     if n != self.m.depot])
52             )
53             for k in sorted_vehicles[:12]:
54                 route = sol[k]
55                 for i in range(1, len(route)):
56                     for j in range(i + 1, len(route) + 1):
57                         test_route = route[:i] + [p] + route[i:j]
58                             + [d] + route[j:]
59                         if self.is_valid_route(test_route):
60                             old_cost = self._calculate_route_
61                                 cost(route)
62                             new_cost = self._calculate_route_
63                                 cost(test_route)
64                             if new_cost - old_cost < min_
65                                 increment:
66                                 min_increment = new_cost - old_
67                                     cost
68                                 best_pos = (k, i, j)
69
70             if best_pos:
71                 k, i, j = best_pos
72                 sol[k].insert(i, p)
73                 sol[k].insert(j, d)
74             else:
75                 k = sorted_vehicles[0]
76                 sol[k].insert(-1, p)
77                 sol[k].insert(-1, d)
78
79     def _calculate_route_cost(self, route: List[int]) -> float:

```

```

71         """Calculate total travel distance of a single route"""
72         if not route:
73             return 0.0
74         cost = self.m.distance[(self.m.depot, route[0])]
75         for i in range(len(route) - 1):
76             cost += self.m.distance[(route[i], route[i+1])]
77         cost += self.m.distance[(route[-1], self.m.depot)]
78         return cost

```

A.3.2 C.2: SA-LNS Algorithm Framework

Listing 2: SA-LNS core framework

```

1  import math
2  import random
3
4  # Weighted objective function (consistent across all algorithms)
5  def objective(solution):
6      return (alpha * total_distance(solution) +
7              beta * total_working_time(solution) +
8              gamma * unserved_requests(solution))
9
10 # Shaw removal destroy operator
11 def shaw_removal(solution, k):
12     selected_requests = select_related_requests(solution, k)
13     partial_solution = remove_requests(solution, selected_
14                                     requests)
15     return partial_solution
16
17 # Regret-k insertion repair operator
18 def regret_repair(partial_solution):
19     while uninsered_requests_exist(partial_solution):
20         request = select_max_regret_request(partial_solution)
21         insert_request_best_position(partial_solution, request)
22     return partial_solution
23
24 # Simulated annealing acceptance criterion
25 def accept_sa(candidate, current, temperature):
26     if objective(candidate) < objective(current):
27         return True
28     delta = objective(candidate) - objective(current)
29     probability = math.exp(-delta / temperature)

```

```

29     return random.random() < probability
30
31 # Main SA-LNS algorithm loop
32 def SA_LNS():
33     current = initial_solution()
34     best = current
35     temperature = INITIAL_TEMPERATURE
36     for iteration in range(MAX_ITERATIONS):
37         partial_solution = shaw_removal(current, removal_size)
38         candidate = regret_repair(partial_solution)
39         if accept_sa(candidate, current, temperature):
40             current = candidate
41         if objective(current) < objective(best):
42             best = current
43         temperature *= COOLING_RATE
44     return best

```

A.4 Appendix D: Minimal ALNS Python Implementation

Listing 3: *Minimal ALNS implementation*

```

1  import math
2  import random
3  from dataclasses import dataclass
4  from typing import Dict, List, Set
5
6  EPS = 1e-9
7  INF = 10**18
8
9  # Core Data Structures
10 @dataclass
11 class Node:
12     node_id: int
13     x: float
14     y: float
15     demand: float
16     ready: float
17     due: float
18     service: float
19     pickup: int
20     delivery: int
21

```

```

22 @dataclass
23 class Request:
24     request_id: int
25     pickup: int
26     delivery: int
27     demand: float
28
29 @dataclass
30 class Solution:
31     routes: List[List[int]]
32     bank: Set[int]
33     def copy(self):
34         return Solution([r[:] for r in self.routes], set(self.
35             bank))
36
37 # Core Evaluation
38 def evaluate_route(route: List[int], inst, capacity):
39     current_time = inst.nodes[0].ready
40     load, feasible = 0.0, True
41     for i, nid in enumerate(route):
42         u = inst.nodes[nid]
43         if i > 0:
44             current_time += inst.travel[route[i-1]][nid]
45             if current_time > u.due + EPS:
46                 feasible = False
47             load += u.demand
48             if load < -EPS or load > capacity + EPS:
49                 feasible = False
50             current_time = max(current_time, u.ready) + u.service
51     return feasible
52
53 def solution_cost(sol, inst):
54     cost = 0.0
55     for r in sol.routes:
56         if not r:
57             continue
58         cost += inst.dist[0][r[0]]
59         for i in range(len(r)-1):
60             cost += inst.dist[r[i]][r[i+1]]
61         cost += inst.dist[r[-1]][0]
62     return cost + 1e5 * len(sol.bank)

```

```

62
63 # Insertion Heuristic
64 def greedy_insert(sol, inst, capacity):
65     while sol.bank:
66         best, min_cost = None, INF
67         for rid in list(sol.bank):
68             req = inst.requests[rid]
69             for r_idx, route in enumerate(sol.routes):
70                 for p in range(len(route)+1):
71                     for d in range(p+1, len(route)+2):
72                         tr = (route[:p] + [req.pickup] + route[p
73                             :d] +
74                             [req.delivery] + route[d:])
75                         if evaluate_route(tr, inst, capacity):
76                             c = solution_cost(Solution([tr], set
77                                 ()), inst)
78                             if c < min_cost:
79                                 min_cost, best = c, (r_idx, p, d
80                                     , rid)
81
82             if best:
83                 r_idx, p, d, rid = best
84                 sol.routes[r_idx].insert(p, inst.requests[rid].
85                     pickup)
86                 sol.routes[r_idx].insert(d, inst.requests[rid].
87                     delivery)
88                 sol.bank.remove(rid)
89             else:
90                 break
91
92 # ALNS Main Loop
93 def alns(inst, vehicle_count, capacity, iterations=100):
94     sol = Solution([[ for _ in range(vehicle_count)], set(inst.
95         requests.keys())])
96     greedy_insert(sol, inst, capacity)
97     best = sol.copy()
98     removal_weights = {"random": 1.0, "shaw": 1.0}
99     T = -(0.05 * solution_cost(sol, inst)) / math.log(0.5)
100
101     for it in range(iterations):
102         cand = sol.copy()
103         q = random.randint(4, min(20, len(inst.requests)//2))

```

```
97     # Random removal
98     planned = [n for r in cand.routes for n in r if inst.
99                 nodes[n].delivery != 0]
100    removed = random.sample(planned, min(q, len(planned)))
101    if planned else []
102    for rid in removed:
103        req = inst.requests[rid]
104        for r in cand.routes:
105            if req.pickup in r:
106                r.remove(req.pickup)
107            if req.delivery in r:
108                r.remove(req.delivery)
109        cand.bank.add(rid)
110
111    # Repair
112    greedy_insert(cand, inst, capacity)
113
114    # SA Acceptance
115    delta = solution_cost(cand, inst) - solution_cost(sol,
116                inst)
117    if delta < 0 or random.random() < math.exp(-delta/(T+EPS
118        ))):
119        sol = cand
120        if solution_cost(sol, inst) < solution_cost(best,
121            inst):
122            best = sol.copy()
123    T *= 0.99975
124    return best
```